

9 Types of API Testing

First, think of API testing as checking whether your backend APIs work correctly, work with other systems, remain reliable under heavy traffic, and are secure.

Let's say, your application has:

POST /api/login

GET /api/users/123

POST /api/appointments

GET /api/appointments

DELETE /api/appointments/123

Now follow 9 types of API testing.

- | | | |
|------------------------|-----------------------|---------------------|
| 1. Smoke Testing | 4. Regression Testing | 7. Security Testing |
| 2. Functional Testing | 5. Load Testing | 8. UI Testing |
| 3. Integration Testing | 6. Stress Testing | 9. Fuzz Testin |

1. Smoke Testing

Purpose: Quickly check whether the API is basically working after a new deployment or build.

Think like "Is the system alive enough to start deeper testing?"

Example:

After deploying your backend, you might test:

</> http

GET /api/health

Expected:

</> JSON

{

 "status": "healthy"

}

Then test:

</> http

POST /api/login

With valid credentials.

If these basic APIs work, you proceed with more detailed testing.

So, what it checks

- API server is running
- Major endpoints respond
- Database connection is working
- Basic authentication works
- No immediate deployment failure

Real-world example

You deploy a Node.js application to AWS.

First, you check:

Application running?

/api/health works?

Login works/

Database reachable?

If /api/health doesn't work, there's little point running hundreds of detailed tests.

Smoke testing = basic "is it alive?" test.

2. Functional Testing

Purpose: To verify that each API does exactly what it is supposed to do.

Think like: "Does this API actually perform its intended functional?"

Suppose you have:

</> http

POST /api/users

Request:

</> JSON

```
{  
  "name": "John",  
  "email": "john@example.com"  
}
```

Expected:

</> http

201 Created

and:

</> JSON

```
{  
  "id": 101,  
  "name": "John",  
  "email": "john@example.com"  
}
```

You test things like:

Valid input

</> Json

```
{  
  "name": "John",  
  "email": "john@example.com"  
}
```

Expected:

201 Created

Missing email

</> JSON

```
{  
  "name": "John"  
}
```

Expected:

400 Bad Request

Duplicate email

Expected something like:

409 Conflict

Invalid email

</> JSON

```
{  
  "name": "John",  
  "email": "hello"  
}
```

Expected:

400 Bad Request

Here, functional testing verifies the business rules of the API.

3. Integration Testing

Purpose: Check whether multiple components/systems work correctly together.

Think: "Do these different pieces communicate correctly?"

An API usually doesn't work alone.

For example:

Frontend -> Backend API -> Authentication -> PostgreSQL -> Payment Service -> Email Service

Suppose:

</> http

POST /api/appointment

Creates an appointment.

The API may need to:

Receive request -> Validate JWT -> Check doctor availability -> Write appointment to Postgre SQL -> Send confirmation email

Integration testing checks this entire interaction.

Example:

You send:

</> http

POST /api/appointments

Then verify:

API -> Database

API -> Authentication

API -> Email services

The individual components might work perfectly by themselves, but integration testing checks whether they work together.

4. Regression Testing

Purpose: Make sure a new change didn't break something that previously worked.

Think: "I changed something. Did I accidentally break something else?"

Suppose your login API previously worked:

POST /api/login

You then modify the authentication system.

You should retest:

Login, logout, get user profile, update profile, create appointment, get appointment, delete appointment.

Why?

Because changing authentication might unexpectedly affect other APIs.

Example: Developer changes – JWT authentication then

Regression testing discovers: Login, user profile, appointments, admin dashboard.

The developer can then fix the issue before production.

Regression testing = "Make sure old functionality still works."

5. Load Testing

Purpose: Determine how the API behaves under an expected amount of traffic.

Think like "Can my API handle normal heavy traffic?"

Suppose your API normally receives:

100 requests/second

But during peak hours you expect:

1,000 requests/second

You simulate:

1,000 requests/sec

And monitor:

- Response time
- CPU
- Memory
- Database connections
- Error rate
- Throughput

For example:

Requests/sec: 1,000

Average response: 150 ms

Error rate: 0.1%

CPU: 65%

That might indicate the system handles expected traffic reasonably well.

Important distinction: load testing uses expected traffic.

Example:

Expected traffic = 1,000 req/sec

Test = 1,000 req/sec

6. Stress Testing

Purpose: Find the breaking point of your API.

Think like "How much traffic can this system take before it starts failing?"

Instead of stopping at 1,000 requests/sec:

1,000

2,000

5,000

10,000

20,000

50,000

Eventually you might discover:

10,000 req/sec -> works

20,000 req/sec -> slower

30,000 req/sec -> errors

40,000 req/sec -> system fails

Now you know approximately where your system's limits are.

Load vs Stress

Testing

Load

Stress

For example:

Normal expected = 5,000 req/sec

Question

Can we handle expected traffic?

What happens beyond our expected traffic?

Load test:

5,000 req/sec

Stress test:

10,000 -> 20,000 -> 30,000 -> 50,000...

7. Security Testing

Purpose: Find vulnerabilities and verify that unauthorized users cannot access protected resources.

Think like "Can someone misuse or attack any API?"

Suppose you have:

</> http

GET /api/admin/users

A normal user should not be able to access it.

You test:

</> http

Authorization: Bearer <normal-user-token>

Expected:

</> http

403 forbidden

But an administrator:

</> http

Authorization: Bearer <admin-token>

Should receive:

</> http

200 OK

Security testing can check

Authentication: Can unauthenticated users access protected APIs?

Authorization: Can a normal user access admin APIs?

JWT/token security: Can an expired token be used?

SQL injection

For example: ' OR '1'='1

Input validation: <script>...</script>

Sensitive data exposure

Check whether APIs accidentally return:

</> JSON

```
{  
  "password": "...",  
  "secretKey": "...",  
  "databasePassword": "...",  
}
```

They shouldn't.

8. UI Testing

Purpose: Test the application through the user interface and verify that the UI correctly communicates with APIs.